
Oliver Krauss

Supervised Machinelearning

AIN2IL | MTD.ba SS2026

Hagenberg 30.04.2026

HAGENBERG | LINZ | STEYR | WELS



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

Questions we Need to Address

- Does learning help in the future, i.e. does experience from previously observed examples help us to solve a future task?
- What is a good model? How do we assess the quality of a model?
- Will a given model be helpful in the future?

Basic Setup: Inputs (Repetition)

Assume we want to learn something about objects from a certain set/space. Most often, sets of such objects are represented by *matrices*:

$$\mathbf{X} = \begin{pmatrix} \mathbf{x}_1^1 & \dots & \mathbf{x}_d^1 \\ \vdots & \ddots & \vdots \\ \mathbf{x}_1^I & \dots & \mathbf{x}_d^I \end{pmatrix}$$

The *rows* in such a matrix correspond to *samples* (I in total) and *columns* correspond to *features/variables* (d in total). The entry $\mathbf{x}_j^i$ (the element in the i -th row and j -th column) corresponds to which value the feature j has for the i -th sample.

If a column consists a finite set of labels, we speak of a *categorical variable/feature*. If a column contains numerical values/numbers, we speak of a *numerical variable/feature*.

Basic Setup: Inputs vs. Outputs

Further assume that we know a target value $y^i \in \mathbb{R}$ for each training sample no. i . All these values constitute the *target/label vector*:

$$\vec{y} = (y^1, \dots, y^l)^T$$

The *training data matrix* is then defined as follows:

$$(X \mid \vec{y}) = \begin{pmatrix} x_1^1 & \dots & x_d^1 & y^1 \\ \vdots & \ddots & \vdots & \vdots \\ x_1^l & \dots & x_d^l & y^l \end{pmatrix}$$

Classification or Regression

Classification the target/label values are categorical, i.e. from a finite set of labels. We will often consider binary classification, i.e. where we have two classes $\rightarrow$ we will use the labels -1 (negative class) and +1 (positive class) or 0 and 1.

Regression the target/label values are numerical

How to Assess Generalization Performance?

The quality of a model can only be judged on the basis of its performance on future data. So assume that future data are generated according to some *joint distribution of inputs and targets*, the *joint density* of which we denote as

$$p(\vec{x}, y)$$

The *generalization error* (or *risk*) is the **expected error on future data** for a given model.

Loss Functions

Assume that the mapping g corresponds to a model which maps the input vector $\vec{x}$ to the predicted output value $g(\vec{x})$.

Then a *loss function*

$$L(y, g(\vec{x}))$$

measures the loss/cost that incurs for a given data sample $(\vec{x}, y)$ (i.e. with real output value y).

The *generalization error* (or *risk*) is the *expected loss on future data* for a given model $g(.)$.

Examples of Loss Functions

Zero-one loss:

$$L_{\mathbf{zo}}(y, g(\vec{x})) = \begin{cases} 0 & y = g(\vec{x}) \\ 1 & y \neq g(\vec{x}) \end{cases}$$

Quadratic loss:

$$L_{\mathbf{q}}(y, g(\vec{x})) = (y - g(\vec{x}))^2$$

Generalization Error/Risk

The *generalization error* (or *risk*) is the expected loss on future data for a given model $\mathbf{g}(\cdot)$:

$$\begin{aligned} R(\mathbf{g}(\cdot)) &= \mathbb{E}(L(\mathbf{y}, \mathbf{g}(\vec{\mathbf{x}}))) \\ &= \int L(\mathbf{y}, \mathbf{g}(\vec{\mathbf{x}})) \cdot p(\vec{\mathbf{x}}, \mathbf{y}) d(\vec{\mathbf{x}}, \mathbf{y}) \end{aligned}$$

The risk for the quadratic loss is called *mean squared error (MSE)*.

Estimating the Generalization Error

Since we typically do not know the distribution $p(\vec{x}, y)$, we have to estimate the generalization performance by making use of already existing data. Two methods are common:

Test set/holdout method: the data set is split randomly into a *training set* and a *test set*; a predictor is trained on the former and evaluated on the latter;

Cross validation: the data set is split randomly into a certain number k of equally sized folds; k predictors are trained, each leaving out one fold as test set; the average performance on the k test folds is computed;

If the samples are independent and identically distributed, then the test set method provides an *unbiased estimate of the performance on future data*. A similar results holds for cross validation.

Five-Fold Cross Validation

	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	
Iteration 1:	Test	Train	Train	Train	Train	→ Perf ₁

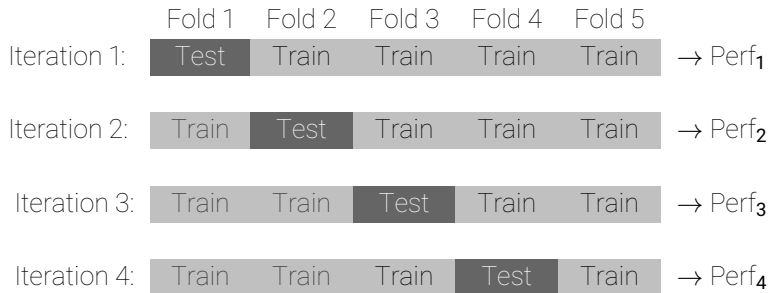
Five-Fold Cross Validation

	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	
Iteration 1:	Test	Train	Train	Train	Train	→ Perf ₁
Iteration 2:	Train	Test	Train	Train	Train	→ Perf ₂

Five-Fold Cross Validation

	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	
Iteration 1:	Test	Train	Train	Train	Train	→ Perf ₁
Iteration 2:	Train	Test	Train	Train	Train	→ Perf ₂
Iteration 3:	Train	Train	Test	Train	Train	→ Perf ₃

Five-Fold Cross Validation



Five-Fold Cross Validation

	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	
Iteration 1:	Test	Train	Train	Train	Train	→ Perf ₁
Iteration 2:	Train	Test	Train	Train	Train	→ Perf ₂
Iteration 3:	Train	Train	Test	Train	Train	→ Perf ₃
Iteration 4:	Train	Train	Train	Test	Train	→ Perf ₄
Iteration 5:	Train	Train	Train	Train	Test	→ Perf ₅

Five-Fold Cross Validation

	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	
Iteration 1:	Test	Train	Train	Train	Train	→ Perf ₁
Iteration 2:	Train	Test	Train	Train	Train	→ Perf ₂
Iteration 3:	Train	Train	Test	Train	Train	→ Perf ₃
Iteration 4:	Train	Train	Train	Test	Train	→ Perf ₄
Iteration 5:	Train	Train	Train	Train	Test	→ Perf ₅

$$\mathbf{CV\ Perf} = \frac{1}{5} \sum_{i=1}^5 \text{Perf}_i$$

Confusion Matrix for Binary Classification

Consider a binary classifier $g(\cdot)$. Then a sample $(\vec{x}, y)$ is called

true positive (TP) if $y = +1$ and $g(\vec{x}) = +1$,

true negative (TN) if $y = -1$ and $g(\vec{x}) = -1$,

false positive (FP) if $y = -1$ and $g(\vec{x}) = +1$,

false negative (FN) if $y = +1$ and $g(\vec{x}) = -1$.

The definitions are analogous if the two classes are 0 and 1 (then 0's are considered *negative* and 1's are considered *positive*).

Confusion Matrix (cont'd)

Given a data set, the *confusion matrix* is defined as follows:

		<i>predicted value $g(x)$</i>	
		+1	-1
<i>actual value y</i>	+1	True Positive	False Negative
	-1	False Positive	True Negative

We will use #TP for True Positives; #FP for False Positives; #FN for False Negatives and #TN for True Negatives

Measures Derived From the Confusion Matrices

Accuracy: proportion of correctly classified items, i.e.

$$\text{ACC} = \frac{\#TP + \#TN}{\#TP + \#FN + \#FP + \#TN}.$$

True Positive Rate (aka recall/sensitivity): proportion of correctly identified positives, i.e.

$$\text{TPR} = \frac{\#TP}{\#TP + \#FN}.$$

False Positive Rate: proportion of negative examples that were incorrectly classified as positives, i.e.

$$\text{FPR} = \frac{\#FP}{\#FP + \#TN}.$$

Positive predictive value (aka precision): proportion of predicted positive examples that were correct, i.e.

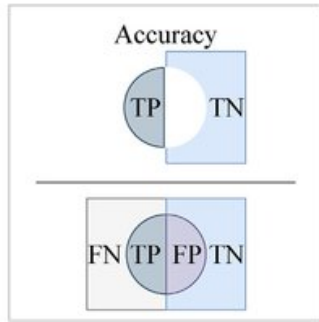
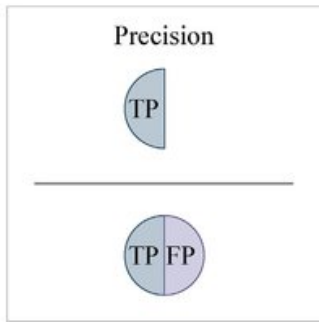
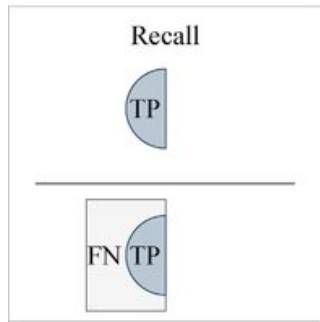
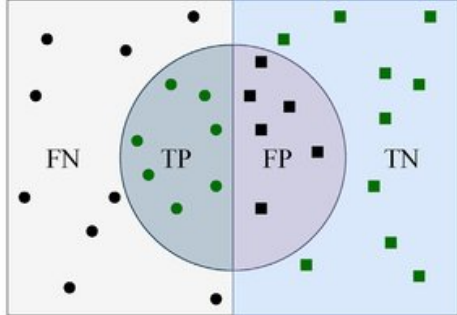
$$\text{PPV} = \frac{\#TP}{\#TP + \#FP}.$$

True Negative Rate (aka specificity): proportion of correctly identified negatives, i.e.

$$\text{TNR} = \frac{\#TN}{\#FP + \#TN}.$$

False Negative Rate: proportion of positive examples that were incorrectly classified as negatives, i.e.

$$\text{FNR} = \frac{\#FN}{\#TP + \#FN}.$$



Evaluation Measures For Unbalanced Data

Balanced Accuracy: mean of true positive and true negative rate, i.e.

$$\text{BACC} = \frac{\text{TPR} + \text{TNR}}{2}.$$

Matthews Correlation Coefficient: measure of non-randomness of classification; defined as normalized determinant of confusion matrix, i.e.

$$\text{MCC} = \frac{\#TP \cdot \#TN - \#FP \cdot \#FN}{\sqrt{(\#TP + \#FP)(\#TP + \#FN)(\#TN + \#FP)(\#TN + \#FN)}}.$$

F-score: harmonic mean of precision and recall, i.e.

$$F_1 = 2 \cdot \frac{\text{PPV} \cdot \text{TPR}}{\text{PPV} + \text{TPR}}.$$

Confusion Matrix for Multi-Class Tasks

Assume that we have a **M** -class classification task. Given a (test) data set, the *confusion matrix* is defined as follows:

		predicted class $g(\vec{x})$				
		1	$\dots$	j	$\dots$	M
actual value y	1	C_{11}	$\dots$	C_{1j}	$\dots$	C_{1M}
	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\ddots$	$\vdots$
	i	C_{i1}	$\dots$	C_{ij}	$\dots$	C_{iM}
	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\ddots$	$\vdots$
	M	C_{M1}	$\dots$	C_{Mj}	$\dots$	C_{MM}

The entries C_{ij} correspond to the numbers of test samples that actually belong to class i and have been classified as j by the classifier $g(.)$.

Accuracy for Multi-Class Tasks

For a multi-class classification task (with the notations as on the previous slide), the *accuracy* of a classifier $g(\cdot)$ is defined as

$$\text{ACC} = \frac{\sum_{i=1}^M C_{ii}}{\sum_{i,j=1}^M C_{ij}} = \frac{1}{I} \cdot \sum_{i=1}^M C_{ii},$$

i.e., not at all surprisingly, as the *proportion of correctly classifier samples*. The other evaluation measures cannot be generalized to the multi-class case in a straightforward way. However, we can define them *per class*.

Other Performance Measures for Multi-Class Classification

Beside accuracy, the other evaluation measures cannot be generalized to the multi-class case in a direct way, but we can easily define them for each class separately. Given a class j , we can define the confusion matrix of class j as follows:

		<i>predicted value</i> $g(\vec{x}; \vec{w})$	
		$= j$	$\neq j$
<i>actual value</i> y	$= j$	True Positive _{j}	False Negative _{j}
	$\neq j$	False Positive _{j}	True Negative _{j}

From this confusion matrix, we can easily define all previously known evaluation measures (for class j).

Balanced Accuracy for Multi-Class Tasks

For a multi-class classification task (with the notations as on the previous slide), the *balanced accuracy* of a classifier $\mathbf{g}(\cdot)$ is defined as

$$\text{BACC} = \frac{1}{M} \cdot \sum_{j=1}^M \text{TPR}_j.$$

General Ranking Performance: ROC Curves

If we have a general discriminant function $\bar{g}$ that maps objects to real values, we can adjust to different asymmetric/unbalanced situations by varying the classification threshold θ (which is by default 0):

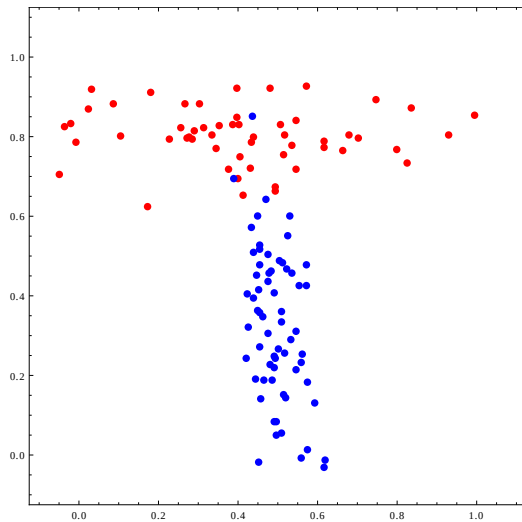
$$g(\vec{x}) = \begin{cases} +1 & \text{if } \bar{g}(\vec{x}) > \theta \\ -1 & \text{otherwise} \end{cases}$$

Receiver-Operator Characteristic (ROC) curves are a simple means for evaluating the performance of a binary classifier *independent of class distributions and misclassification costs*. The basic idea of ROC curves is to plot the true positive rate (TPR) vs. the false positive rate (FPR) while varying the classification threshold. Then the *area under the ROC curve (AUC)* can serve as a measure of general ranking performance.

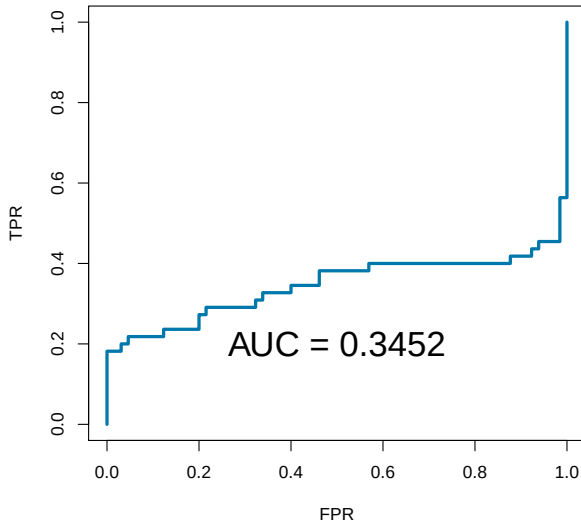
ROC Curves: Practical Realization

- Sort samples descendingly according to $\bar{g}$.
- Divide horizontal axis into as many bins as there are negative samples; divide vertical axis into as many bins as there are positive samples.
- Start curve at **(0, 0)**.
- Iterate over all possible thresholds, i.e. all possible “slots” between two discriminant function values. Every positive sample is a step up, every negative sample is a step to the right.
- In case of ties (equal discriminant function values), process them at once (which results in a ramp in the curve).
- Finally, end curve in **(1, 1)**.

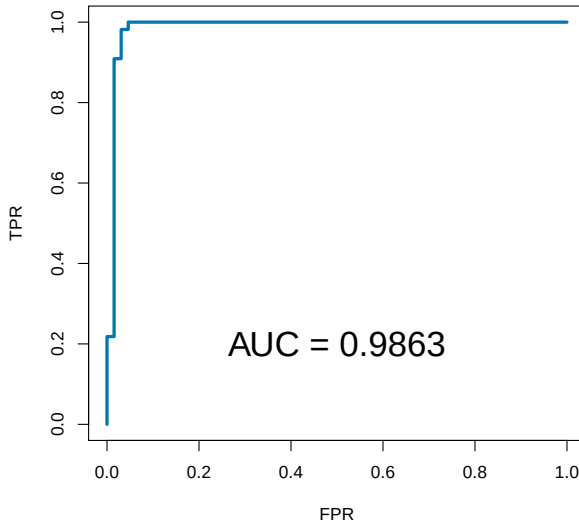
ROC Example



ROC Curve for $g((x_1, x_2)) = x_1$



ROC Curve for $g((x_1, x_2)) = x_2$



Evaluation Measures for Regression I

Given a regression function $g(\cdot)$ and a data set $(\vec{x}^i, y^i) \mid i = 1, \dots, l$, we can define the following evaluation measures.

(Root) Mean Squared Error:

$$\text{MSE} = \frac{1}{l} \sum_{i=1}^l (g(\vec{x}^i) - y^i)^2 \qquad \text{RMSE} = \sqrt{\text{MSE}}$$

Mean Absolute Error:

$$\text{MAE} = \frac{1}{l} \sum_{i=1}^l |g(\vec{x}^i) - y^i|$$

Evaluation Measures for Regression II

Mean Absolute Percentage Error:

$$\text{MAPE} = \frac{100}{I} \sum_{i=1}^I \frac{|g(\vec{x}^i) - y^i|}{y^i}$$

Coefficient of Determination:

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}},$$

where

$$SS_{\text{res}} = \sum_{i=1}^I (g(\vec{x}^i) - y^i)^2, \quad SS_{\text{tot}} = \sum_{i=1}^I (y^i - \bar{y})^2, \quad \bar{y} = \frac{1}{I} \sum_{i=1}^I y^i.$$

Finally, it is also common to compute the *correlation* of predicted values $(g(\vec{x}^1), \dots, g(\vec{x}^I))$ and true values $(y^1, \dots, y^I)$ (mostly using the standard *Pearson correlation coefficient*).

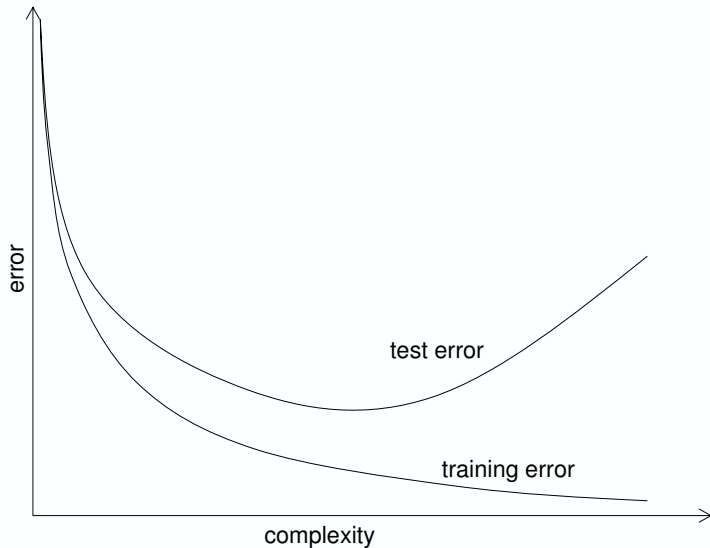
Underfitting and Overfitting

Underfitting: our model is too coarse to fit the data (neither training nor test data); this is usually the result of too restrictive model assumptions (i.e. *too low complexity of model*).

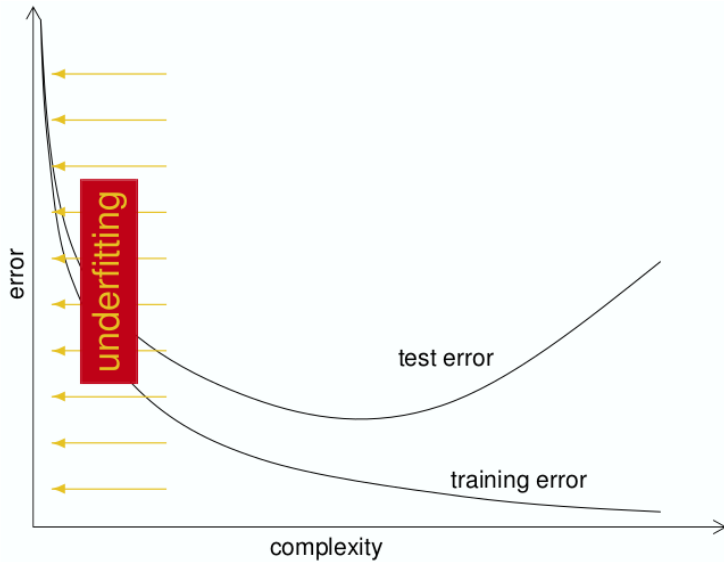
Overfitting: our model works very well on training data, but generalizes poorly to future/test data; this is usually the result of *too high model complexity*.

The best generalization performance is obtained for the optimal choice of the complexity level. An estimate of the optimal choice can be determined by (cross) validation.

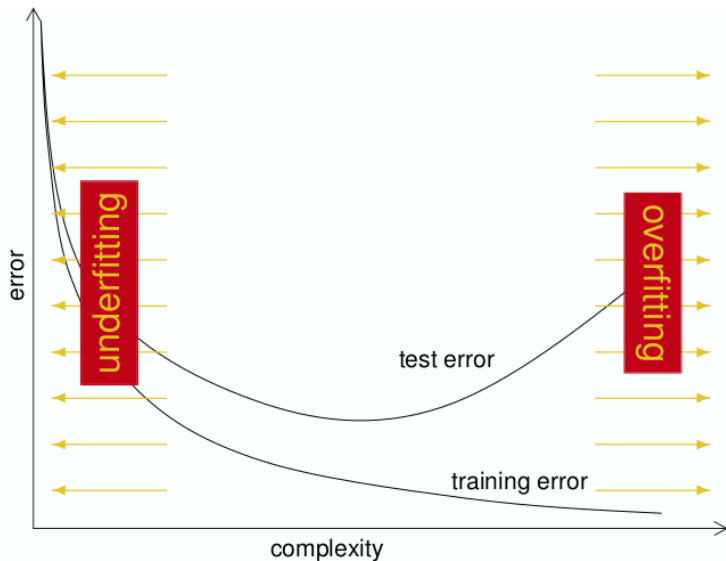
Underfitting and Overfitting



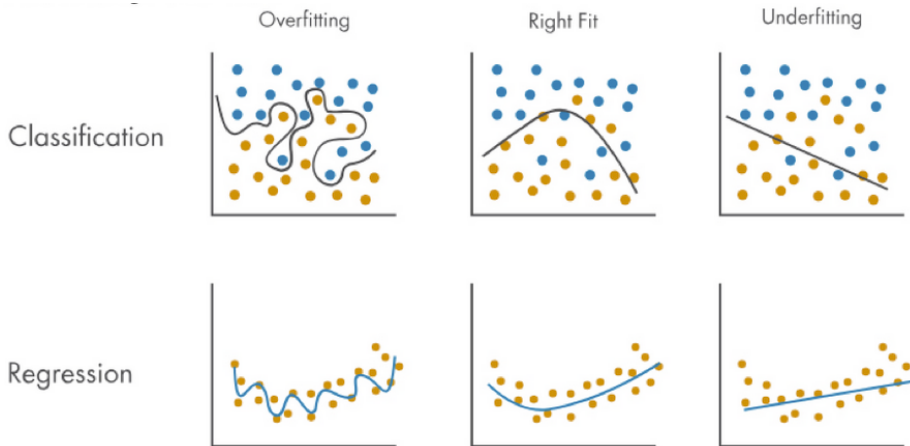
Underfitting and Overfitting



Underfitting and Overfitting



Underfitting and Overfitting



Overfit has a high **variance** (low bias); Underfit has a high **bias** (low variance)

Hyperparameter Optimization

- Usually, hyperparameters cannot be optimized during training, but need to be optimized outside of the actual training procedure by estimating the generalization error.
- In order to perform hyperparameter optimization, we need two ingredients:
 1. Evaluation scheme: how to estimate generalization performance
 2. Optimization scheme: how to find optimal hyperparameters

Hyperparameter Optimization: Evaluation Schemes

Test set/regular cross validation Possible in principle, but no samples left for estimating the final generalization performance. (hyperparameters are biased to entire data set!)

Validation set Use partition into three sets:

1. Train parameters on training set.
2. Estimate generalization performance on validation set to find best hyperparameters.
3. Once the best hyperparameters have been found, evaluate generalization performance of final model on test set.

Double/nested cross validation Perform hyperparameter optimization in inner CV loop and estimation of generalization performance in outer CV loop.

Hybrid Perform hyperparameter optimization using CV on training set and estimate generalization performance on test set.

Hyperparameter Optimization: Optimization Schemes

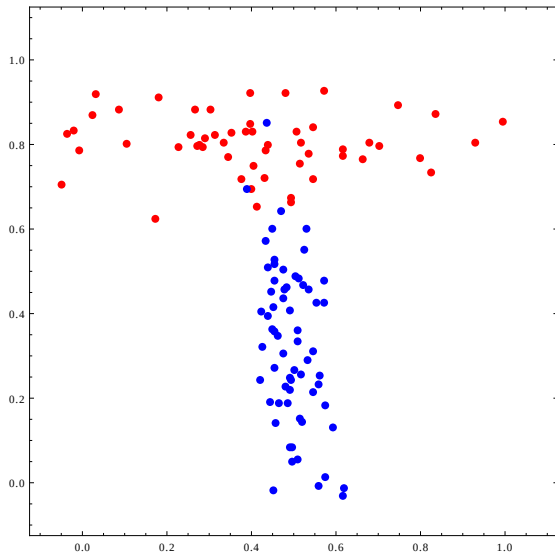
Grid search Discretize range of each hyperparameter into finite grid, consider all possible combinations, and select the combination that gives the best generalization performance.

Random search Sample random combinations of hyperparameters (define sampling distributions for each hyperparameter) and select the combination that gives the best generalization performance.

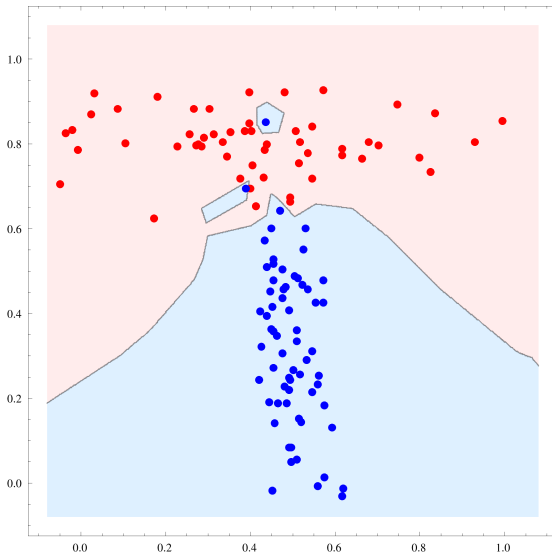
Real optimization methods

- Bayesian optimization: use a probabilistic surrogate of the objective function and update it incrementally
- Gradient methods: use derivatives (if available)
- Evolutionary optimization
- Other heuristic optimization methods

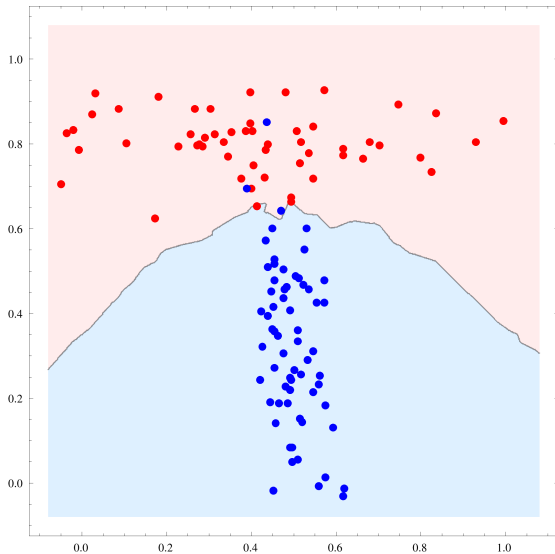
k-Nearest Neighbor Classifier Ex.1



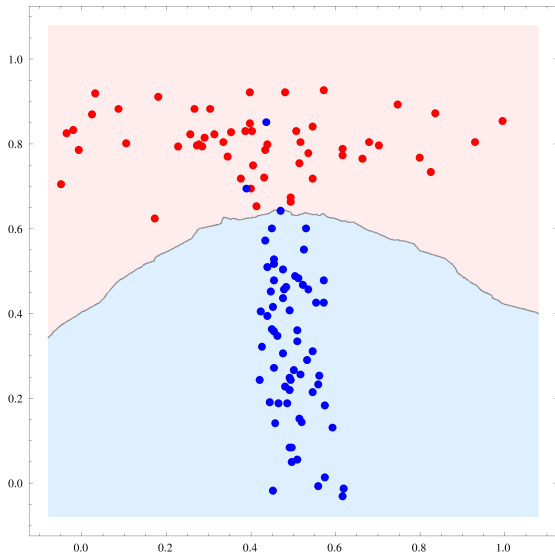
k-Nearest Neighbor Classifier Ex.1 $k = 1$:



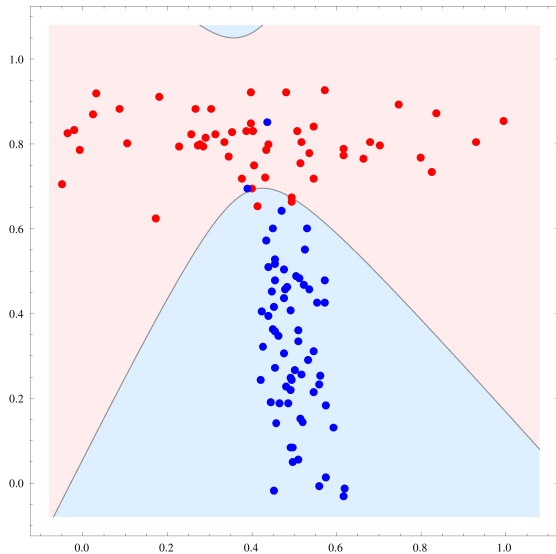
k-Nearest Neighbor Classifier Ex.1 $k = 5$:



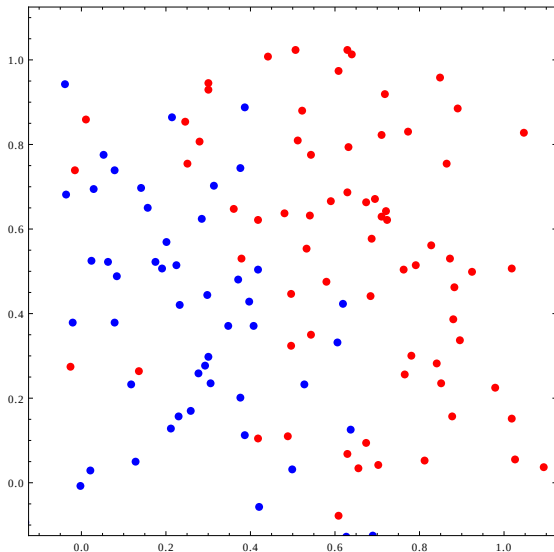
k-Nearest Neighbor Classifier Ex.1 $k = 13$:



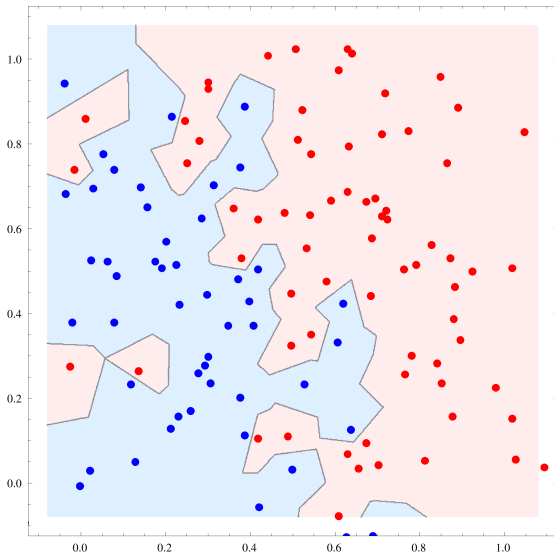
k-Nearest Neighbor Classifier Ex.1 **optimal**



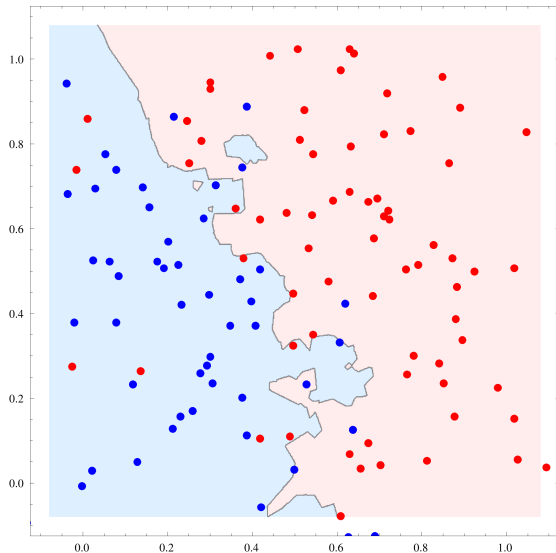
k-Nearest Neighbor Classifier Ex.2



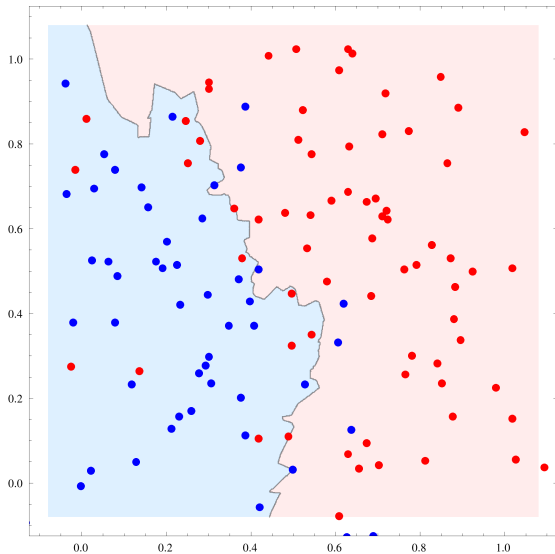
k-Nearest Neighbor Classifier Ex.2 $k = 1$:



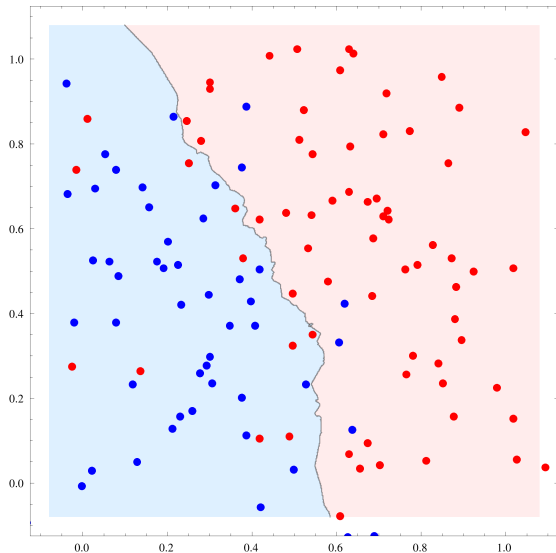
k-Nearest Neighbor Classifier Ex.2 $k = 5$:



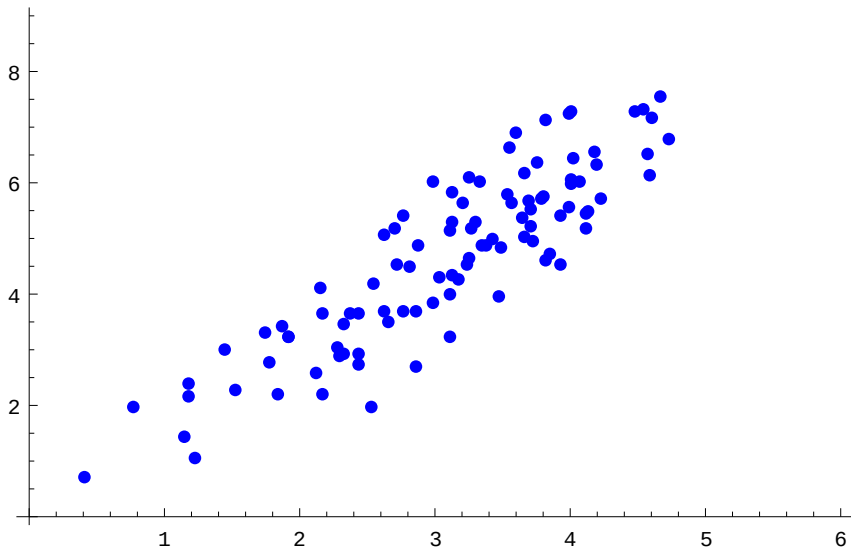
k-Nearest Neighbor Classifier Ex.2 $k = 13$:



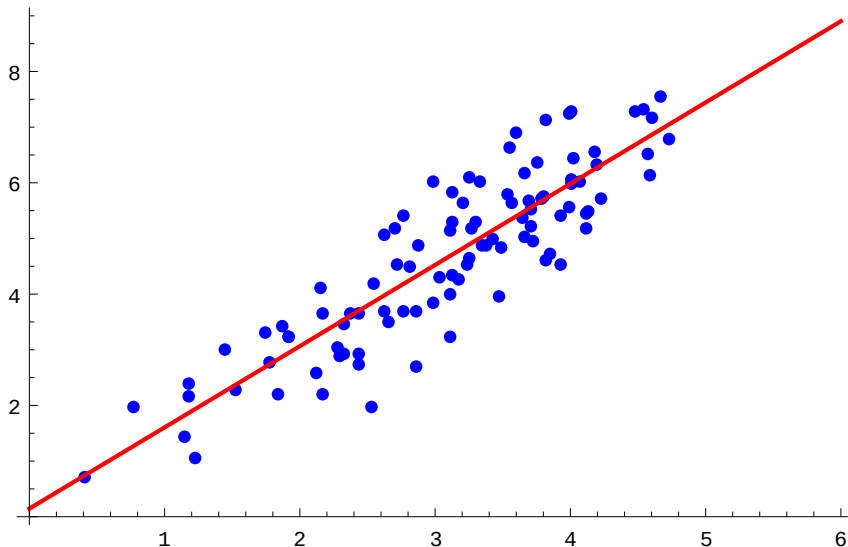
k-Nearest Neighbor Classifier Ex.2 $k = 25$:



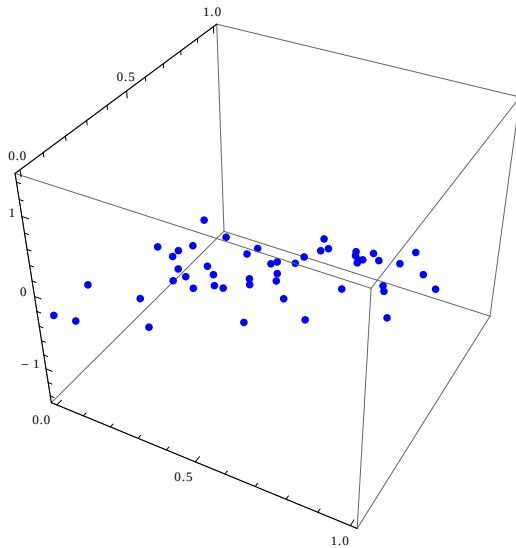
Linear Regression Ex.1



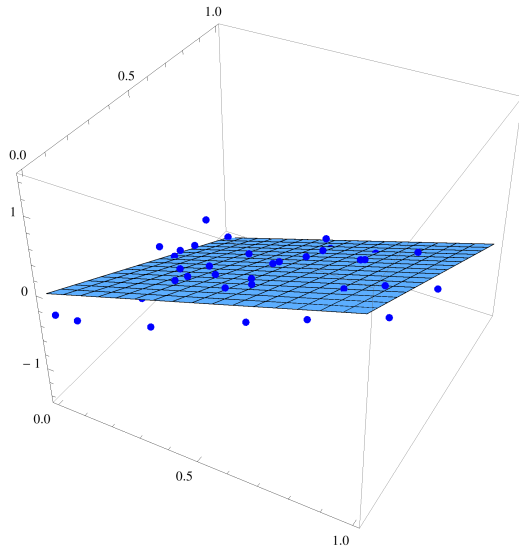
Linear Regression Ex.1



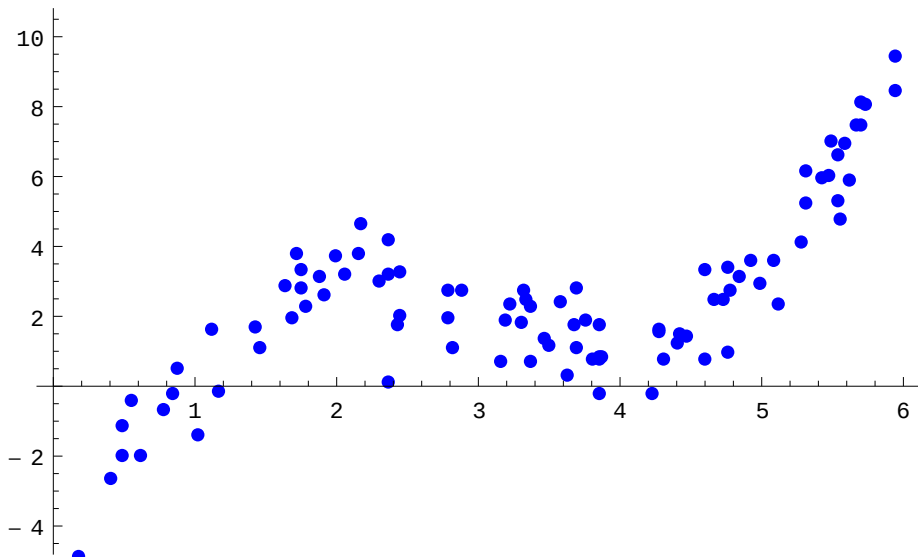
Linear Regression Ex.2



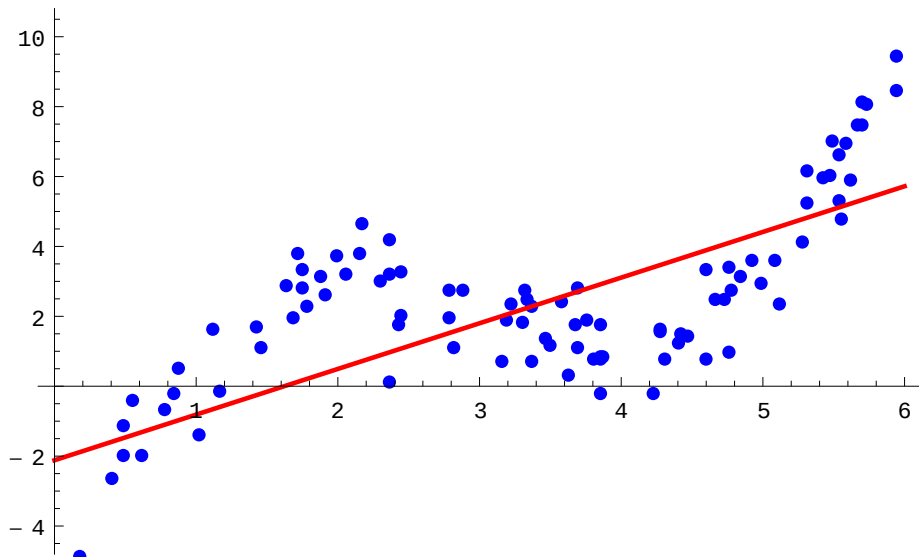
Linear Regression Ex.2



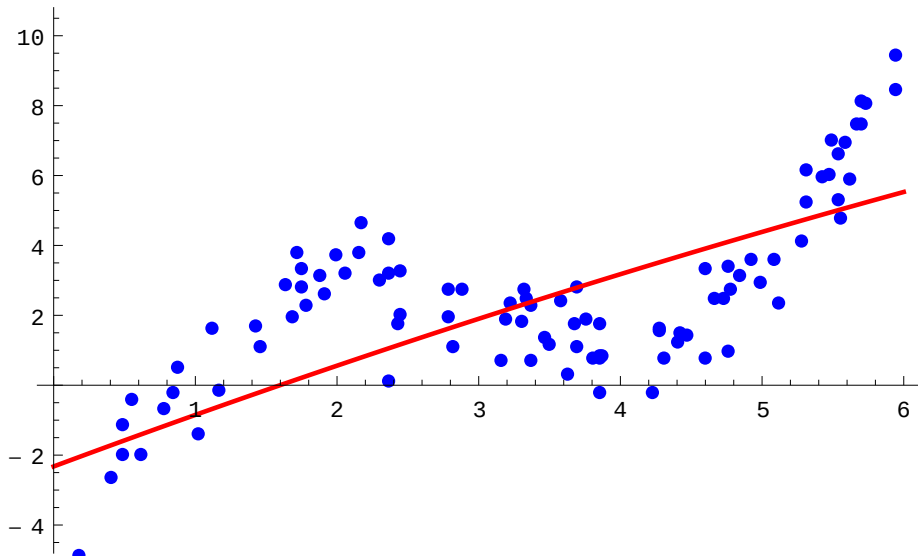
Polynomial Regression Ex.



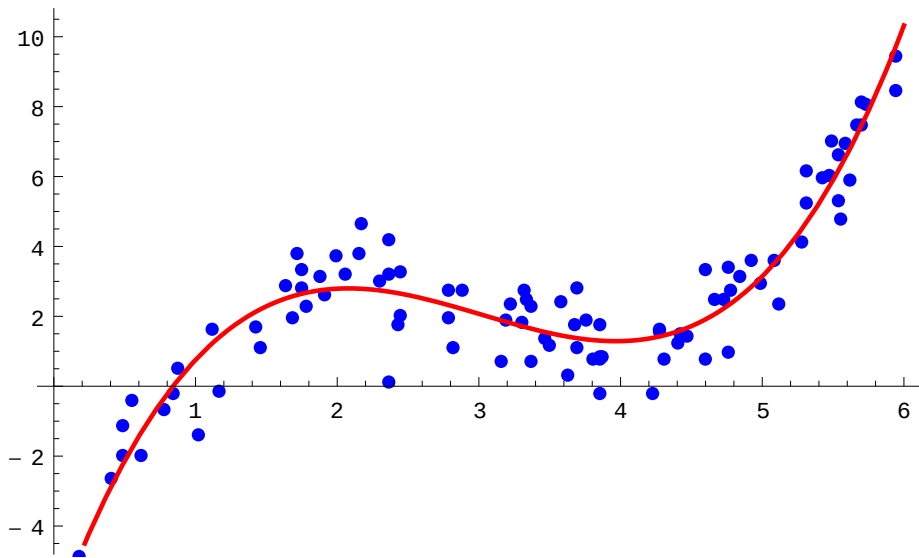
Polynomial Regression Ex. $n = 1$:



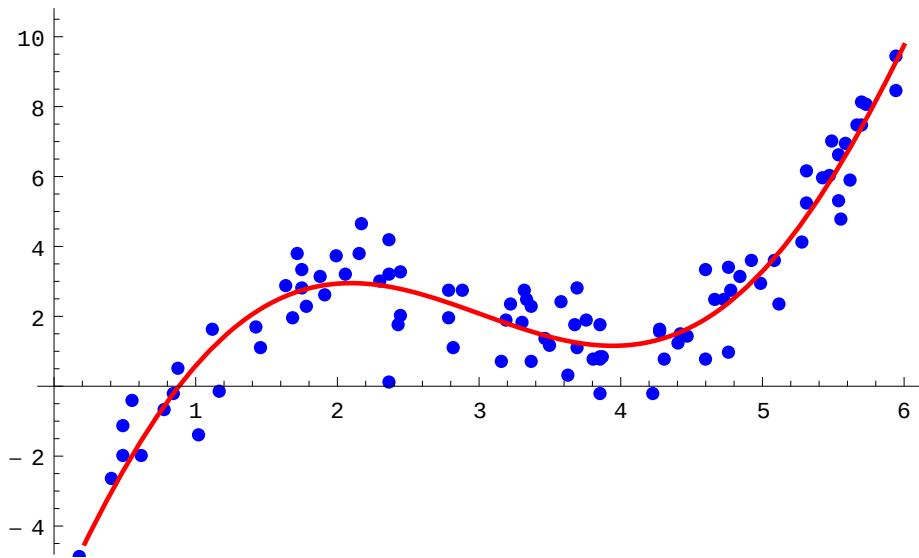
Polynomial Regression Ex. $n = 2$:



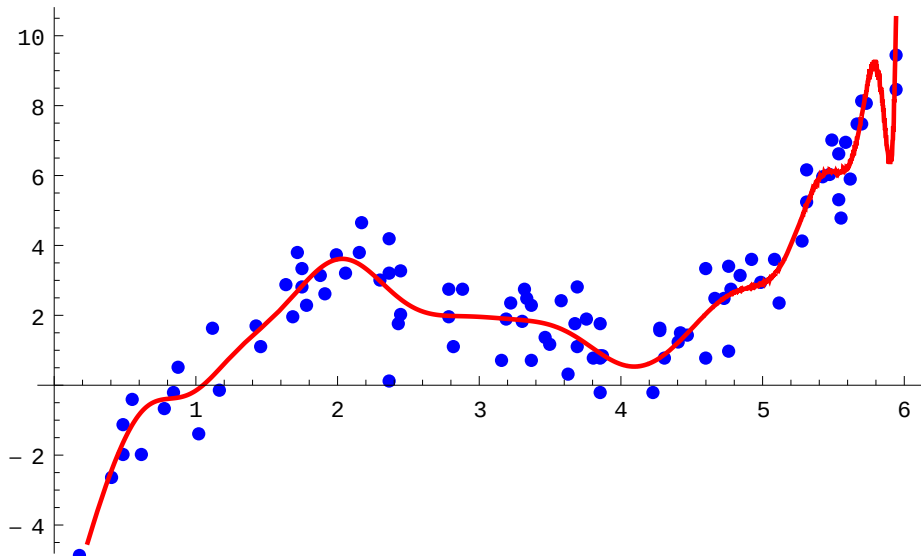
Polynomial Regression Ex. $n = 3$:



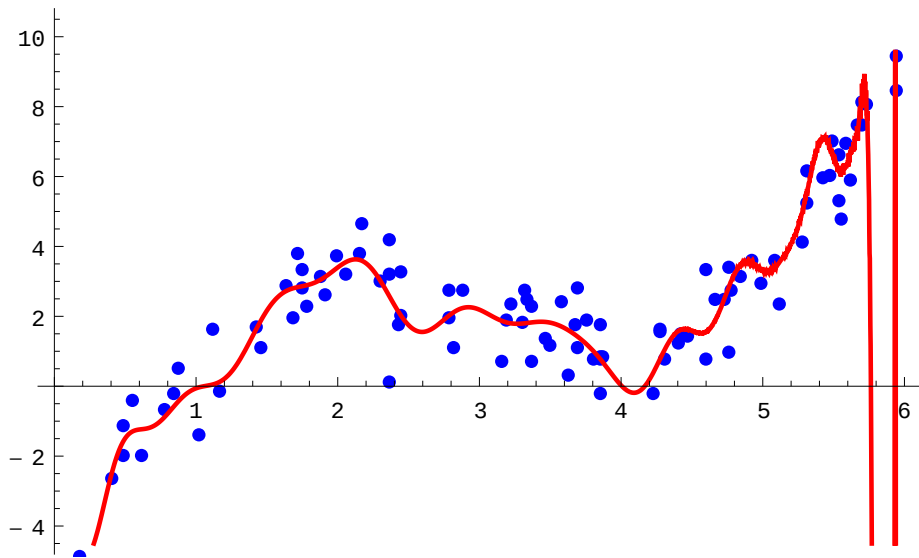
Polynomial Regression Ex. $n = 5$:



Polynomial Regression Ex. $n = 25$:



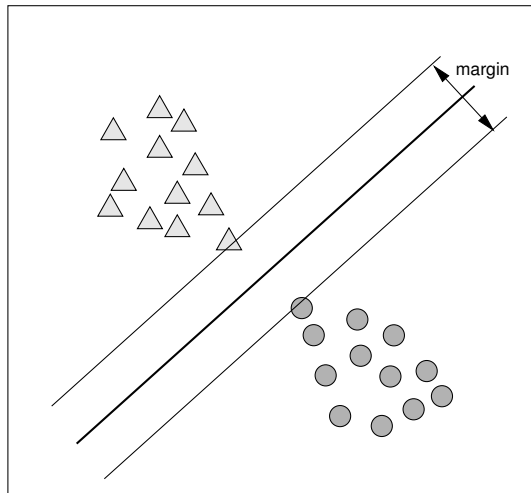
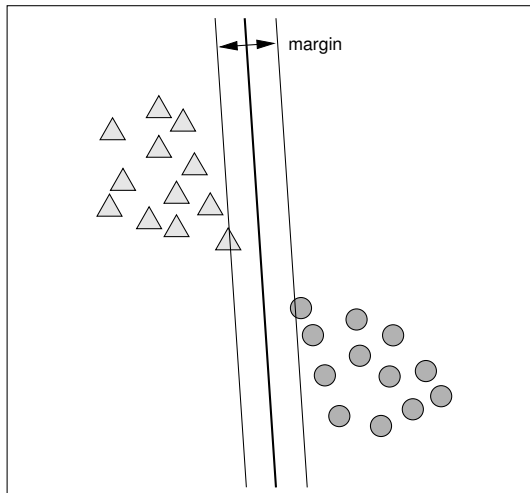
Polynomial Regression Ex. $n = 75$:



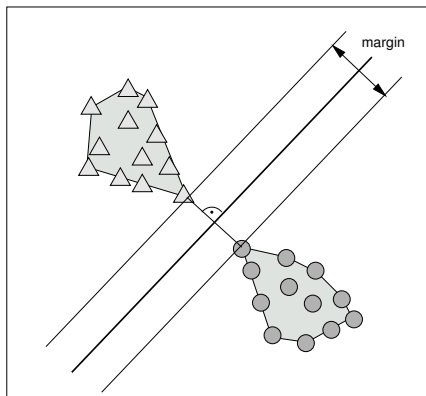
Support Vector Machines in a Nutshell

- Simply, *Support Vector Machines (SVMs)* are based on the idea of finding a classification border that maximizes the margin between positive and negative samples.
- According to a theoretical result, maximizing the margin corresponds to minimizing an upper bound of the generalization error.

Margin Maximization I



Margin Maximization II



The two classes are linearly separable if and only if their convex hulls are disjoint.

If the two classes are linearly separable, margin maximization can be achieved by making an orthogonal 50:50 split of the shortest distance connecting the convex hulls of the two classes.

The question remains how to solve margin maximization computationally: by quadratic optimization.

SVM Discriminant Function

For a given training set $\{(\vec{x}^i, y^i) \mid 1 \leq i \leq I\}$, a common *support vector machine classifier* is represented as the discriminant function

$$g(\vec{x}) = b + \sum_{i=1}^I \alpha_i \cdot y^i \cdot k(\vec{x}, \vec{x}^i),$$

where b is a real value, α_i are non-negative factors, and $k(.,.)$ is the so-called *kernel*, a similarity measure for the inputs. The discriminant function only depends on those samples whose Lagrange multiplier α_i is not 0. Those are called *support vectors*.

Standard Kernels

Linear: $k(\vec{x}, \vec{y}) = \vec{x} \cdot \vec{y}$

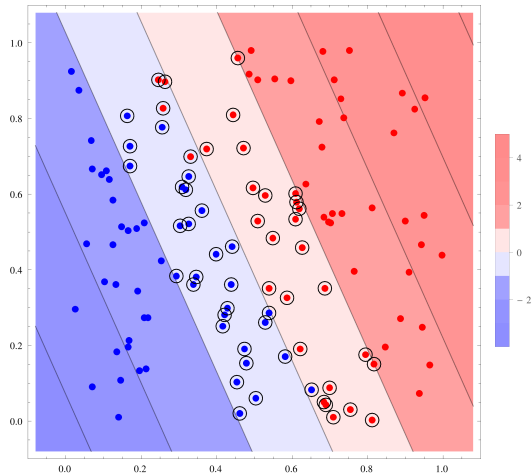
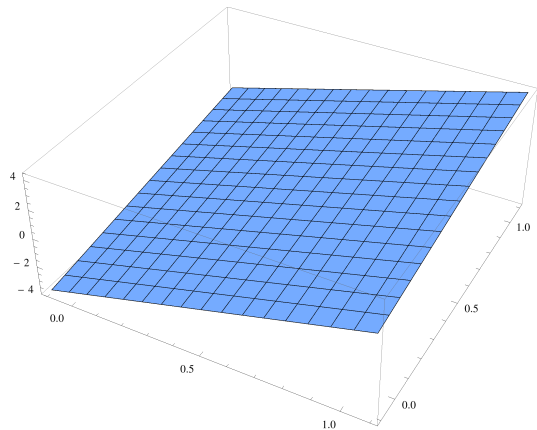
Polynomial: $k(\vec{x}, \vec{y}) = (\vec{x} \cdot \vec{y} + \beta)^\alpha$

Gaussian/RBF¹: $k(\vec{x}, \vec{y}) = \exp \left(- \underbrace{\frac{1}{2\sigma^2}}_{=\gamma} \|\vec{x} - \vec{y}\|^2 \right)$

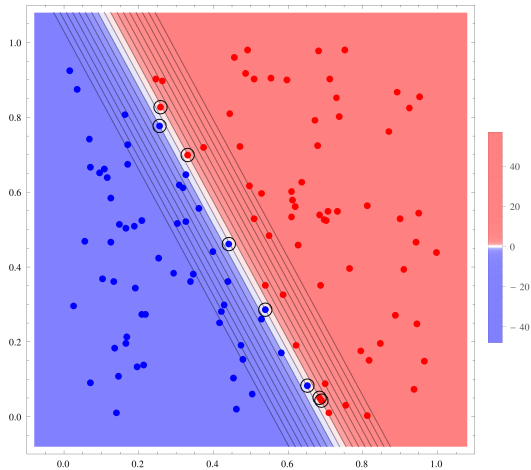
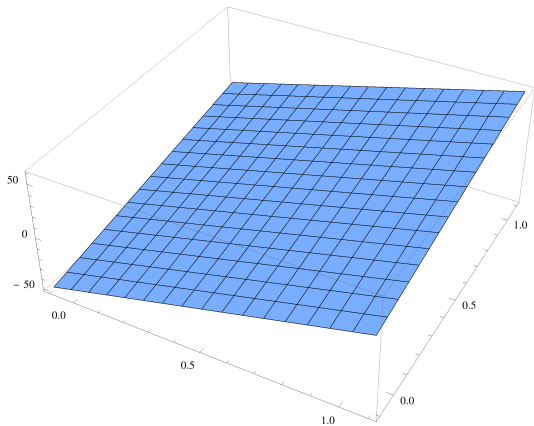
Sigmoid: $k(\vec{x}, \vec{y}) = \tanh(\alpha \vec{x} \cdot \vec{y} + \beta)$

¹RBF = Radial Basis Function

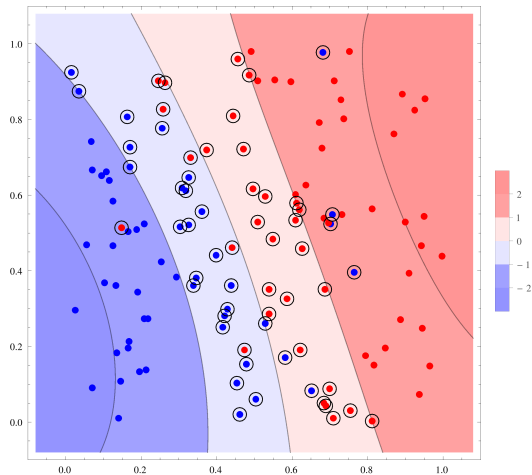
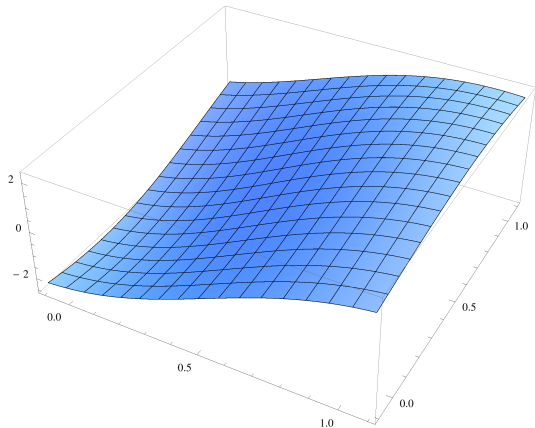
Example: Linear Kernel / $C = 1$



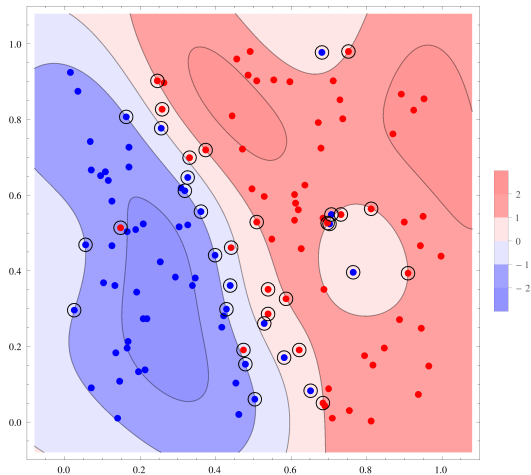
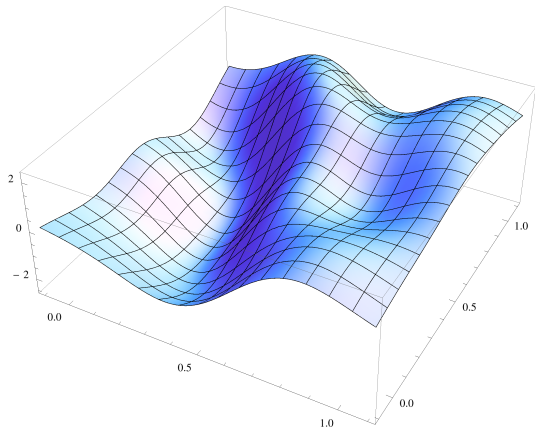
Example: Linear Kernel / $C = 1000$



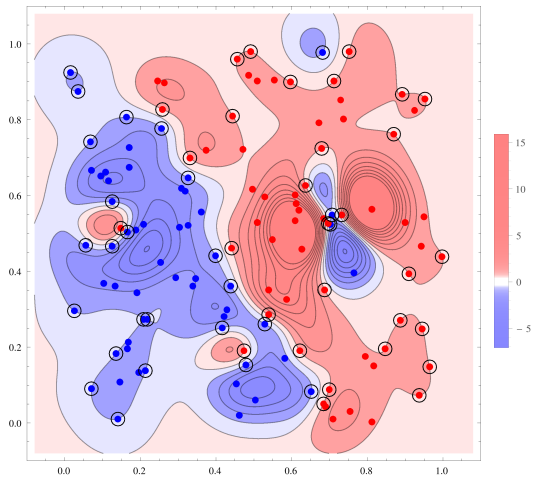
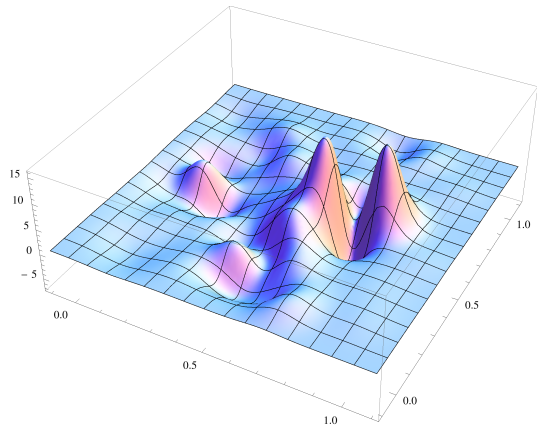
Example: RBF Kernel / $\mathbf{C} = \mathbf{1} / \gamma = \frac{1}{2\sigma^2} = 1$



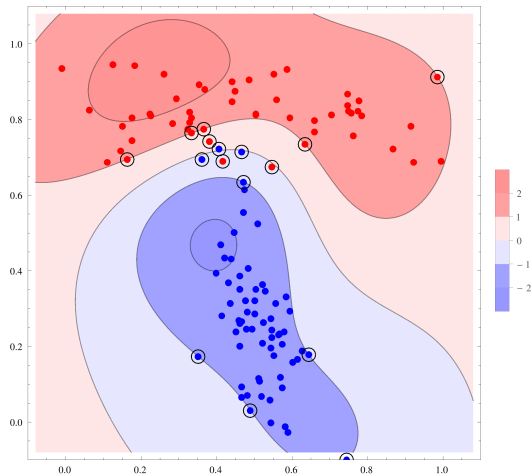
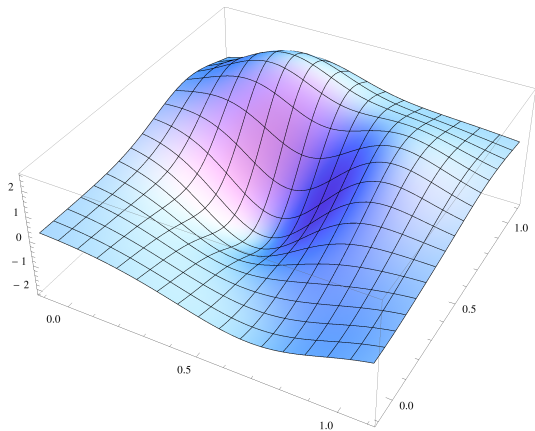
Example: RBF Kernel / $C = 10$ / $\gamma = \frac{1}{2\sigma^2} = 10$



Example: RBF Kernel / $C = 1000$ / $\gamma = \frac{1}{2\sigma^2} = 100$



Example: RBF Kernel / $C = 10$ / $\gamma = \frac{1}{2\sigma^2} = 10$



SVMs for Multi-Class Problems I

- Support vector machines are intrinsically based on the idea of separating two classes by maximizing the margin between them. So there is no obvious way to extend them to multi class problems.
- All approaches introduced so far are based on breaking down the multi-class problem into several binary classification problems.

SVMs for Multi-Class Problems II

Suppose we have a classification problem with M classes.

One against the rest: M support vector machines are trained, where the i -th SVM is trained to distinguish between the i -th class and other classes; a new sample is assigned to the class whose SVM has the highest discriminant function value.

Pairwise classification: $\frac{M(M-1)}{2}$ SVMs are trained, one for each pair of classes; a new sample is assigned to the class that received the most votes from the $\frac{M(M-1)}{2}$ SVMs. This is the better and more common approach.

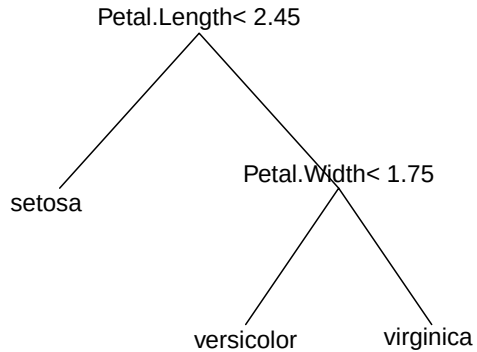
Decision Trees

- A decision tree is a classifier that classifies samples “by asking questions successively”; each non-leaf node corresponds to a question, each leaf corresponds to a final prediction.
- Decision tree learning is concerned with partitioning the training data hierarchically such that the leaf nodes are hopefully homogeneous in terms of the target class.
- Decision trees have mainly been designed for categorical data, but they can also be applied to numerical features.
- Decision trees are traditionally used for classification (binary and multi-class), but regression is possible, too.

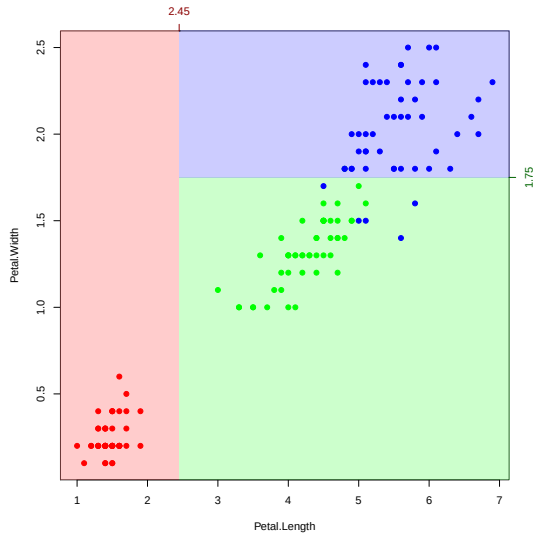
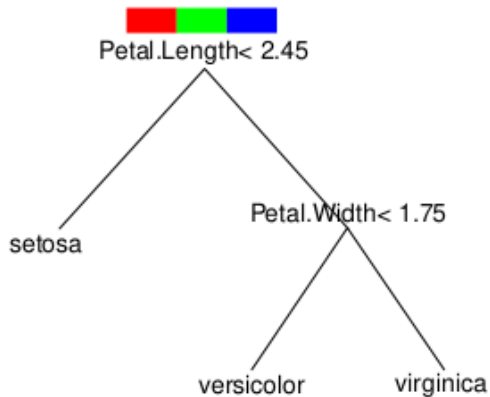
Decision Tree Learning

- All decision tree learning algorithms are recursive, depth-first search algorithms that perform hierarchical splits.
- There are three main design issues:
 1. Splitting criterion: which splits to choose?
 2. Stopping criterion: when to stop further growing of the tree?
 3. Pruning: whether/how to collapse unnecessarily deep sub-trees?
- 2 and 3 are especially relevant for adjusting the complexity of decision trees (underfitting vs. overfitting).

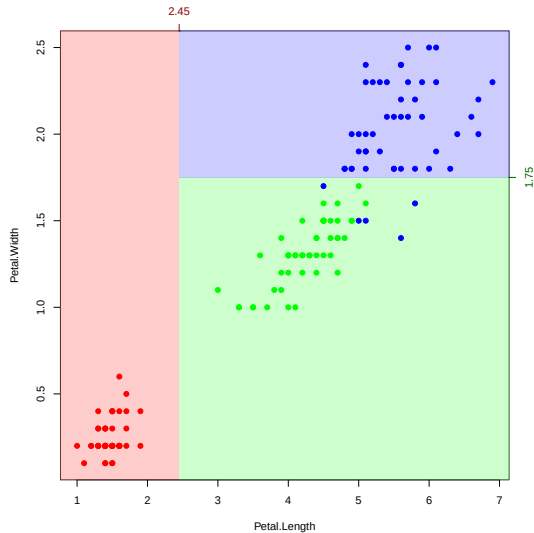
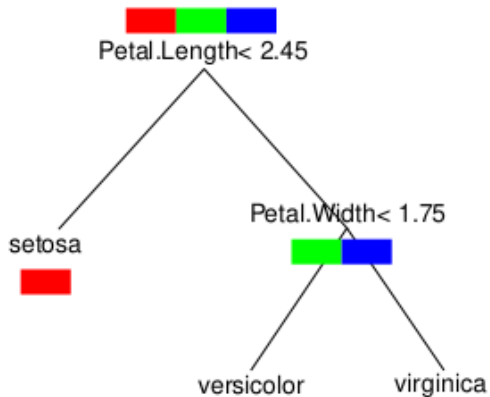
Example: Iris Data Set



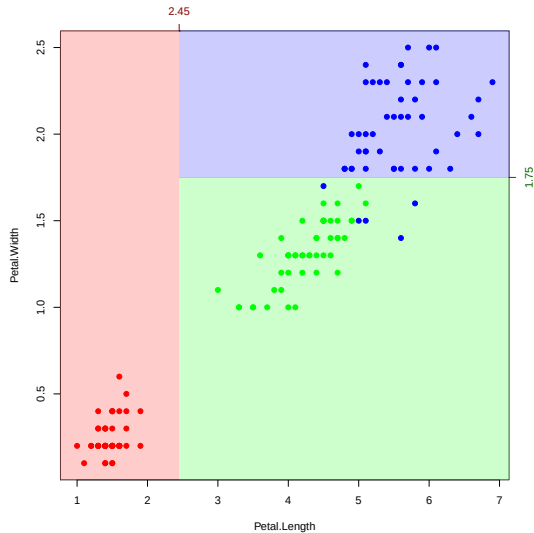
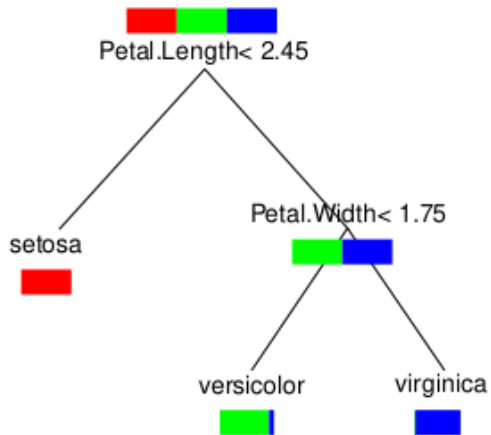
Example: Iris Data Set



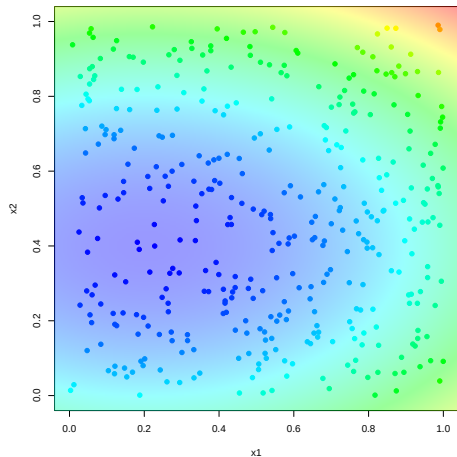
Example: Iris Data Set



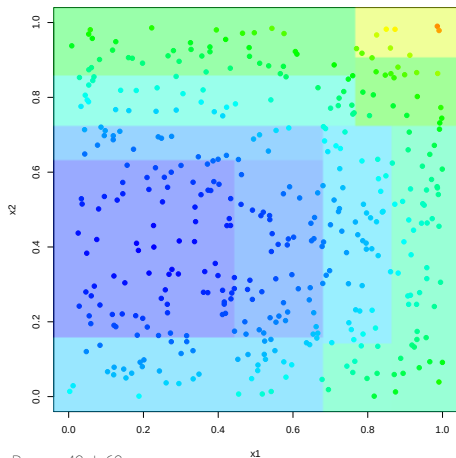
Example: Iris Data Set



Example: Regression Data Set



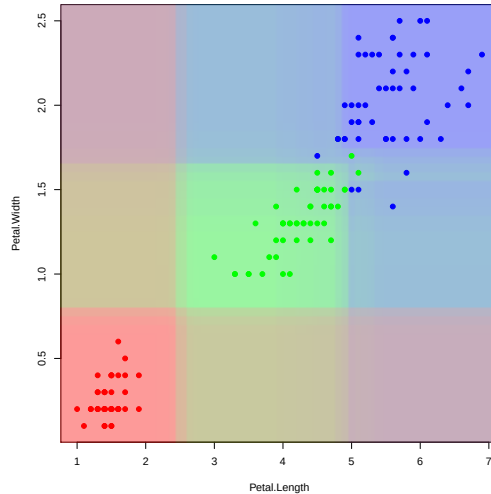
Example: Regression Data Set



Random Forests: Most Common Variant

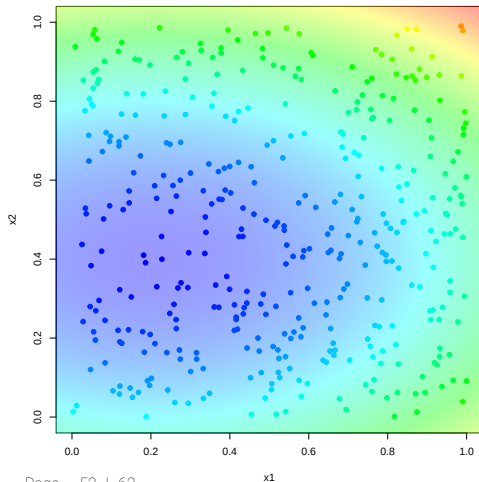
- Use CART (Classification and Regression Trees) for training the single trees, i.e. binary splits with Gini impurity gain (for classification) / variance reduction (for regression) as splitting criterion.
- For each tree, samples are chosen randomly from the training set (typically with replacement).
- For each split, only a sub-sample of randomly chosen features is considered.
- Trees are grown to full size and not pruned.
- Random forests are ensembles using the bagging (bootstrapping + aggregating) approach.

Example: Iris Data Set (1000 trees)

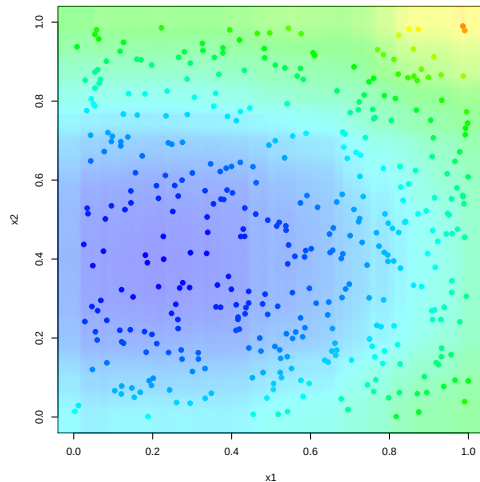


Example: Regression Data Set

True function:



Prediction (1000 trees):



Out-of-Bag Estimates

- Random forests allow for assessing the generalization performance on the basis of training data only.
- For each sample, the error can be computed by considering only those trees that have not used this sample in their training sub-sample.
- Then the overall out-of-bag error can be computed by averaging the out-of-bag errors of all samples.

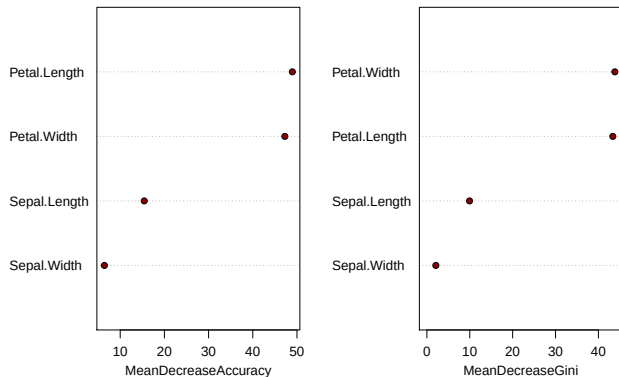
Measuring Variable Importance

Mean Gini impurity decrease: for all features, average the Gini impurity gains of all splits in all trees that involve this feature;

Mean Gini impurity decrease:

1. Compute out-of-bag error for each sample.
2. For each feature separately, consider random permutations and compute the out-of-bag errors for the data set with the permuted feature.
3. Then the importance score is computed by averaging the differences before and after permuting the feature (upon normalization by the standard deviation of the differences).

Example: Variable Importances for Iris Data Set (1000 trees)



Example: Predicting Mortality After Heart Valve Surgeries

Data set: 137 pre-operative parameters of 2,229 patients undergoing heart valve surgeries at former AKH Linz + mortality data

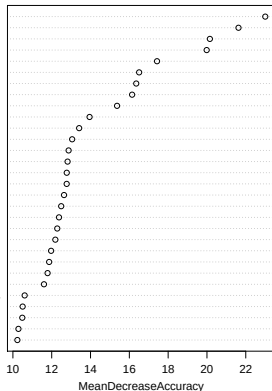
Final approach: predict 30-days mortality with random forest, consider prediction as indication for high risk

Results:

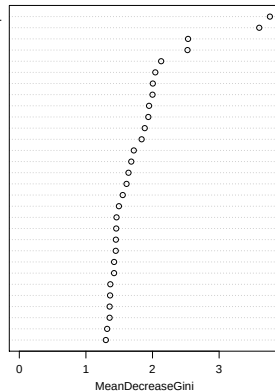
- Patients predicted to have high risk indeed have a significantly higher risk to die (about 20% within first 30 days).
- Much better result than with established risk scores Euroscore I and II.
- Kidney function parameters seem to be most indicative proxies for the general condition of the patient.

Example: Variable Importances for Predicting Mortality After Heart Valve Surgeries

Phosphor..mmol.l.
AST..GOT..U.l.
Leukozyten..G.l.
ABE..mmol.l.
FHHb....
Last.Creatinine.Level.Preop
Gesamt.CK..U.l.
Chlorid..mmol.l.
ALT..U.l.
CRP..mg.dl.
GFR.nach.aMDRD..ml.min.1.73m..
Previous.CV.interventions
Bronchodilators
Magnesium..mmol.l.
Kalium..mmol.l.
KH.Hauptdiagnosen.Code.R57
Classification...NYHA
alk..Phosph...U.l.
Hämoglobin..g.dl.
Thrombozyten..G.l.
MCH..pg.
Previous.Valve
Renal.Failure
Natrium..mmol.l.
Cholesterin..mg.dl.
Kategorie.der.operativen.Leistung.4
KH.Hauptdiagnosen.Code.I21
Kreatinin..mg.dl.
Erythrozyten..T.l.
MCHC..g.dl.

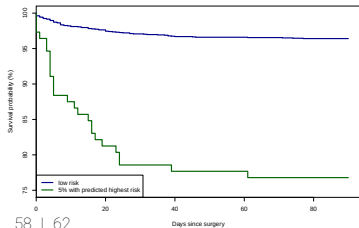
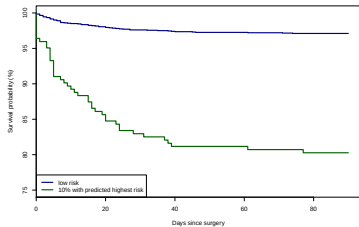


GFR.nach.aMDRD..ml.min.1.73m..
pro.BNP..pg.ml.
Kreatinin..mg.dl.
BUN..mg.dl.
Last.Creatinine.Level.Preop
Erythrozyten..T.l.
Gewicht
Größe
Hämoglobin..g.dl.
Myoglobin..ug.l.
Calcium..mmol.l.
Hämatokrit...
Leukozyten..G.l.
Alter.bei.Aufnahme.a
ABE..mmol.l.
AST..GOT..U.l.
Antithrombin...
Glucose..mg.dl.
Cholesterin..mg.dl.
Thrombozyten..G.l.
Phosphor..mmol.l.
alk..Phosph...U.l.
CRP..mg.dl.
Chlorid..mmol.l.
Magnesium..mmol.l.
BMI
MCV..fl.
MCHC..g.dl.
g.GT..U.l.
Fibrinogen..mg.dl.



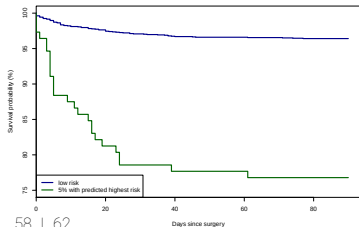
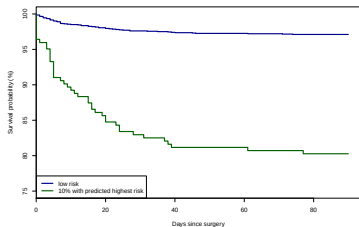
Survival over Time

Random Forest:

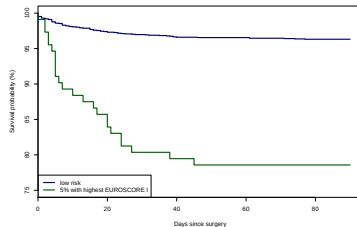
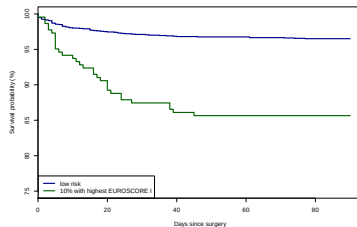


Survival over Time

Random Forest:



Euroscore I:



Ensembles: **Bagging** vs. Boosting

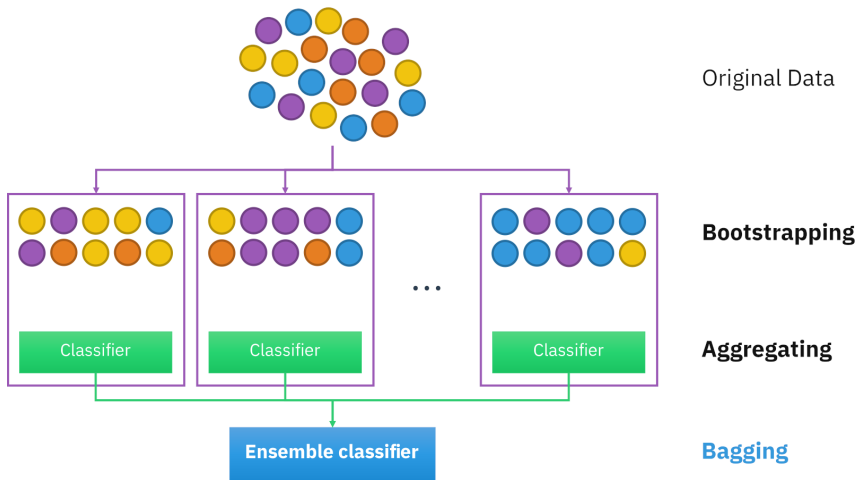


Figure: Source: Sirakorn, CC BY-SA 4.0, Wikimedia Commons

Ensembles: Bagging vs. **Boosting**

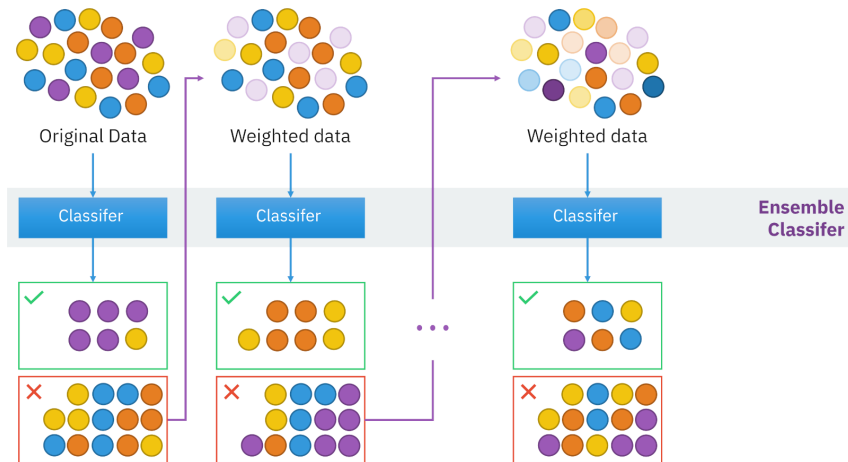


Figure: Source: Sirakorn, CC BY-SA 4.0, Wikimedia Commons

Gradient Tree Boosting

- Random forests are ensembles using the so-called *bagging* (*bootstrapping* + *aggregating*) approach: trees are learned independently and contribute equally.
- *Boosting* is an alternative approach: trees are learned iteratively; in each step, a tree is learned in order to improve the current result.
- **Basic setup:** A decision tree can be written as a piecewise function:

$$h_j(\vec{x}) = \sum_{i=1}^{N_j} \gamma_{ij} \cdot \mathbf{1}_{ij}(\vec{x})$$

N_j is the number of terminal nodes of the j -th tree and $\mathbf{1}_{ij}(\vec{x})$ is an indicator function that yields 1 if $\vec{x}$ “falls into” the i -th terminal node and 0 otherwise.

Gradient Tree Boosting: Basic Procedure

- Then we create a sequence of models $g_m(\vec{x})$ in an iterative fashion by adding trees:

$$g_m(\vec{x}) = g_{m-1}(\vec{x}) + h_m(\vec{x}) = g_{m-1}(\vec{x}) + \sum_{i=1}^{N_j} \gamma_{im} \cdot \mathbf{1}_{im}(\vec{x})$$

- In each step, γ_{im} are optimized using gradient descent to minimize a loss function based on the residual $y - g_{m-1}(\vec{x})$.
- It is common to use quite simple/shallow trees for gradient tree boosting.
- **XGBoost** is a common software library that provides many different variants and customizations.